

Introdução ao Pensamento Computacional

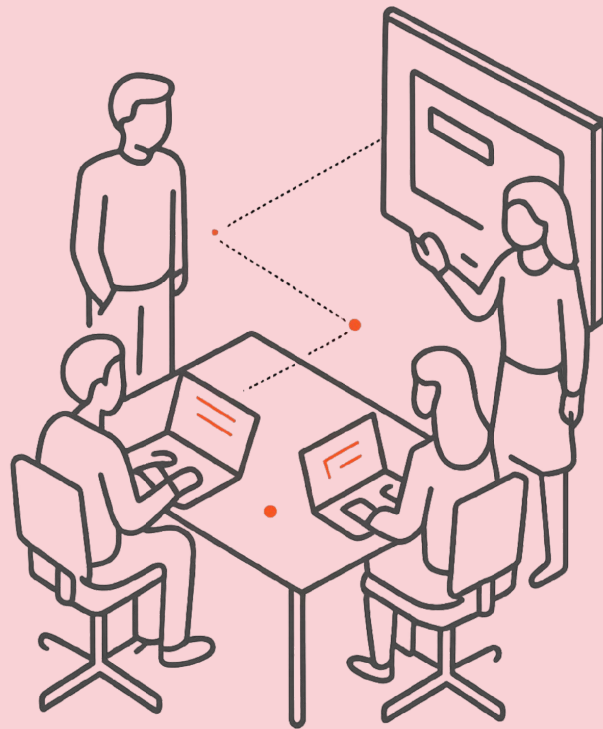
UFPE - Centro de Informática - IEC - 2026.1

DURAÇÃO: 4 HORAS

GOOGLE BLOCKLY



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



OBJETIVO DA AULA

Apresentando o Objetivo

O que aprenderemos hoje

Pensamento Computacional e conceitos básicos de lógica de programação aplicados a problemas interdisciplinares reais.

Como faremos isso

Construiremos fluxos lógicos e soluções através do Google Blockly, compreendendo que os pilares do pensamento computacional fazendo práticas para melhor aprendizagem.

 Reforçar o ambiente colaborativo e seguro: erros de lógica são etapas naturais da modelagem, não falta de talento.

O que Iremos Aprender?

1

Compreender os 4 Pilares

Decomposição, Reconhecimento de Padrões, Abstração e Pensamento Algorítmico.

2

Traduzir Problemas Reais

Converter problemas analíticos do cotidiano em etapas lógicas e pseudo-códigos.

3

Programar em Blocos

Desenvolver códigos funcionais no Google Blockly, sem a carga cognitiva da sintaxe.

4

Entender, Analisar e Testar

Entender a sintaxe da linguagem de programação, analisar o algoritmos, e mitigar as frustração com o erro.

PRÁTICA

Faça Agora!

Desafio Inicial: Como fazer um cuscuz?

Acesse o link através do QRCode ao lado e crie uma publicação descrevendo em linguagem natural a sequência lógica de passos para a solução, mapeando:

1

Entrada

O que entra no processo?

2

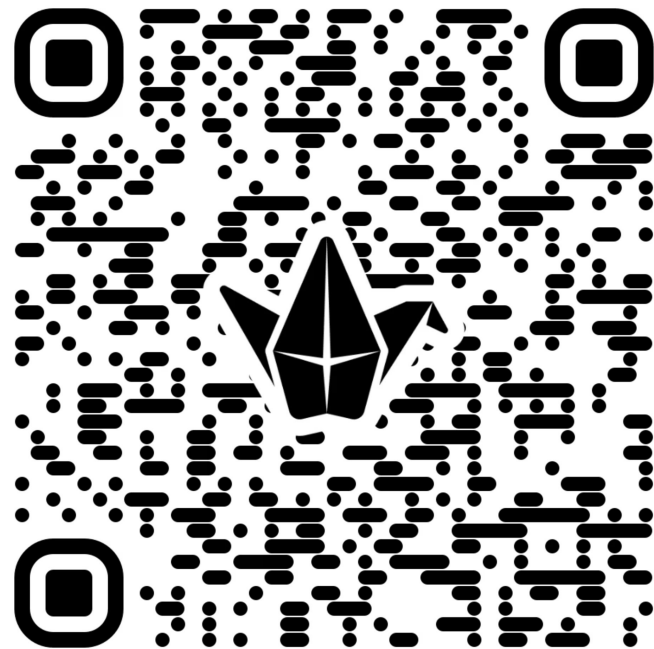
Processo

Como será executado?

3

Saída

Qual é o resultado esperado?



Vamos analisar o que vocês construíram

Após a tentativa de descrever a receita do cuscuz, vamos discutir a experiência em grupo:



Dificuldades

Quais foram os principais obstáculos encontrados ao tentar detalhar cada passo para o "computador"?



Pontos Ambíguos

Que partes da descrição ficaram abertas a diferentes interpretações ou careceram de clareza?



Acertos

Onde a lógica aplicada foi precisa e o "algoritmo", mesmo que inicial, se mostrou eficaz?

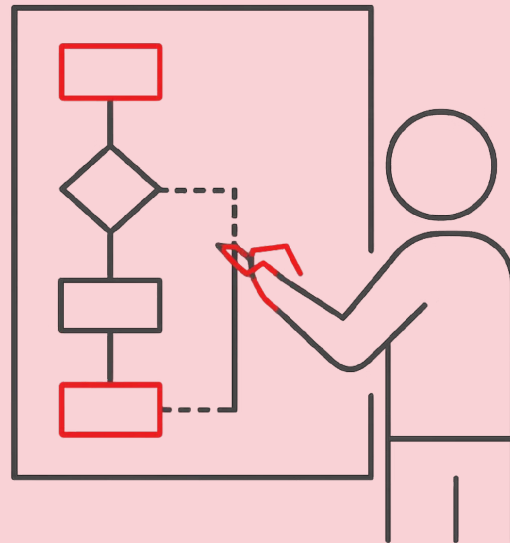
PENSAMENTO COMPUTACIONAL

O que é Pensamento Computacional?

Seus passos fazem sentido para um computador — ou só para você?

Pensamento Computacional é a habilidade de formular problemas e expressar soluções de forma tão clara que qualquer pessoa — ou máquina — consiga executá-las. Não é sobre programar: é sobre pensar com precisão.

— Jeannette Wing, 2006



PENSAMENTO COMPUTACIONAL

Errar Faz Parte do Método

O erro de lógica não é uma falha de talento — é uma etapa natural da modelagem científica e computacional.

Ao analisar as etapas do processo computacional como parte do processo de elaboração dos nossos algoritmos, é comum usarmos a técnica de "tentativa e erro" aleatória e os aproximamos do método sistemático de investigação, o mesmo que guia suas pesquisas acadêmicas.

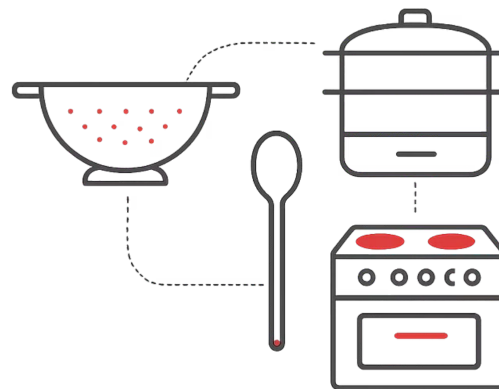
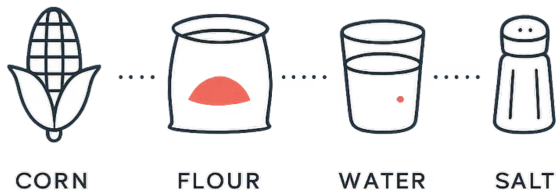


O Algoritmo do Cuscuz



1. Entradas

Para o computador, toda receita começa com **entradas**: os dados e recursos que o algoritmo precisa para funcionar.



1. Ingredientes

Dados primários:

- Flocão de milho (ou farinha de milho flocada)
- Água
- Sal

2. Utensílios

Recursos de hardware/ambiente:

- Tigela (para misturar)
- Cuscuzeira
- Colher
- Fogão (fonte de calor)

O Algoritmo do Cuscuz



2. Processamento

No processamento, a ordem importa: cada passo depende do anterior — como em qualquer algoritmo.

01

Colocar flocão na tigela

Colocar a quantidade desejada de flocão de milho na tigela.

02

Adicionar sal e misturar

Adicionar sal a gosto e misturar os ingredientes secos.

03

Adicionar água aos poucos

Adicionar água aos poucos à tigela, misturando com a colher.

04

Condição: verificar a textura

Repetir o passo 3 até que a massa esteja úmida e com textura de areia molhada.

05

Descansar a massa

Deixar a massa descansar por cerca de 10 minutos (tempo de hidratação).

06

Colocar água na cuscuzeira

Colocar água no compartimento inferior da cuscuzeira (até a marca limite).

07

Encaixar a grelha separadora

Encaixar a grelha separadora na cuscuzeira.

08

Transferir a massa hidratada

Transferir a massa hidratada para a parte superior da cuscuzeira. Não apertar a massa; apenas espalhar levemente para o vapor circular.

09

Tampar a cuscuzeira

Tampar a cuscuzeira.

10

Colocar no fogão

Colocar a cuscuzeira no fogão e acender o fogo (fogo médio).

11

Condição de parada

Aguardar o cozimento. Quando a água ferver e a massa exalar o cheiro característico de cuscuz cozido (cerca de 10 minutos), o processo está concluído.

12

Desligar o fogo

Desligar o fogo.



3. Saída: Produto Final

Um cuscuz cozido, macio, pronto para ser servido no prato.

Os 4 Pilares do Pensamento Computacional

Para abordar problemas de forma eficaz, aplicamos um conjunto de habilidades fundamentais, conhecidas como os Quatro Pilares do Pensamento Computacional. Estas são as ferramentas que nos permitem estruturar o raciocínio e traduzir desafios complexos em soluções claras e executáveis.



Decomposição

Quebrar um problema complexo em partes menores e mais gerenciáveis, tornando cada parte mais fácil de resolver individualmente.



Reconhecimento de Padrões

Identificar similaridades, tendências ou regularidades dentro e entre os problemas. Isso ajuda a prever e reutilizar soluções.



Abstração

Focar nos detalhes importantes e ignorar os irrelevantes para criar um modelo simplificado, generalizando princípios.



Algoritmos

Desenvolver uma sequência de passos lógicos, instruções claras e ordenadas para resolver o problema ou completar uma tarefa.

Decomposição: Desvendando o Quebra-Cabeça

É a arte de quebrar um problema grande e complicado em partes menores e mais simples. Assim, cada pedacinho se torna fácil de resolver individualmente, como montar um LEGO peça por peça.

Exemplo Ludico: Fazer um bolo de cenoura

PROBLEMA: Fazer Bolo de Cenoura

SUB-PROBLEMA 1: Preparar a massa

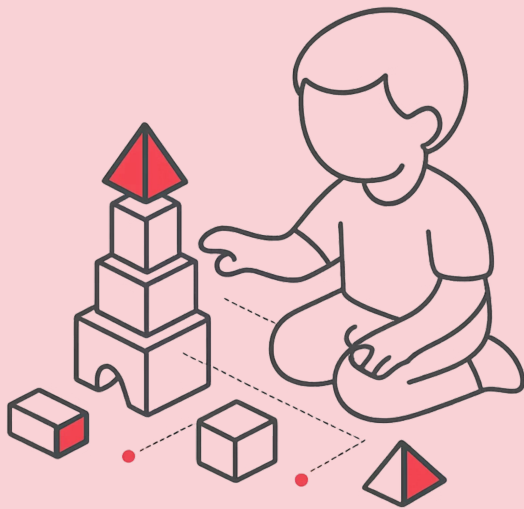
- Misturar ingredientes secos
- Misturar ingredientes úmidos
- Combinar as misturas

SUB-PROBLEMA 2: Preparar a cobertura

- Derreter chocolate
- Misturar com creme de leite

SUB-PROBLEMA 3: Assar e montar

- Despejar massa na forma
- Levar ao forno
- Despejar cobertura sobre o bolo



Abstração: Focando no Essencial

Abstração é a arte de simplificar. É a habilidade de concentrar-se nos detalhes cruciais de um problema, ignorando os aspectos irrelevantes ou menos importantes. Ao fazer isso, criamos um modelo mais limpo e compreensível, que facilita a análise e a busca por soluções.

Exemplo Lúdico: Navegar em uma Cidade

PROBLEMA: Encontrar o caminho mais rápido para um destino em uma cidade desconhecida.

ABSTRAÇÃO: Usar um mapa da cidade.

- Foca em ruas, avenidas, pontos de ônibus, estações de metrô e grandes marcos.
- Ignora detalhes como a cor dos carros, o número exato de pedestres ou a arquitetura de cada prédio.
- Permite planejar a rota de forma eficiente, sem se perder em informações desnecessárias.

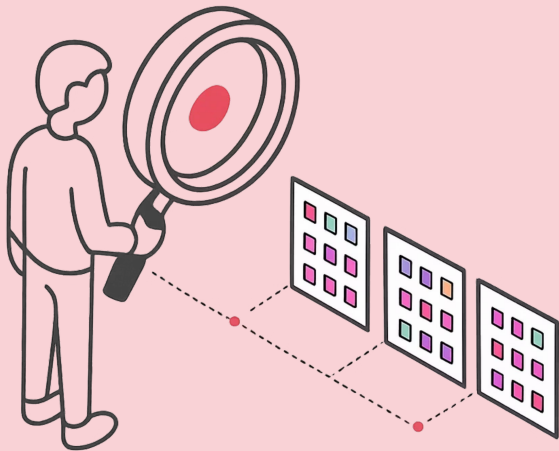


Reconhecimento de Padrões: Desvendando a Repetição

Esta habilidade nos permite identificar semelhanças, tendências e regularidades em dados ou problemas. Ao reconhecer que um problema é similar a algo que já resolvemos, podemos aplicar soluções conhecidas, economizando tempo e esforço. É como notar que a estrutura de uma frase se repete em várias línguas.

Exemplo Lúdico: Encontrando Itens Repetidos na Feira

```
lista_de_compras = ["maçã", "banana", "laranja", "maçã", "pera",  
"banana"]  
itens_repetidos = []  
  
for item in lista_de_compras:  
    if lista_de_compras.count(item) > 1 and item not in itens_repetidos:  
        itens_repetidos.append(item)  
  
print(f"Itens que você anotou mais de uma vez: {itens_repetidos}")  
# Saída esperada: Itens que você anotou mais de uma vez: ['maçã',  
'banana']
```



Algoritmos: A Receita para o Sucesso

Algoritmos são como receitas detalhadas: uma sequência finita de instruções claras e não ambíguas para resolver um problema ou executar uma tarefa. Eles são a espinha dorsal de qualquer processo computacional, garantindo que o "computador" (seja ele uma máquina ou uma pessoa) saiba exatamente o que fazer, quando e como.

Exemplo Lúdico: Atravessar a Rua com Segurança

PROBLEMA: Atravessar a rua com segurança.

PASSO 1: Ir até a faixa de pedestres mais próxima.
PASSO 2: Olhar para a esquerda em busca de veículos.
PASSO 3: Olhar para a direita em busca de veículos.
PASSO 4: Olhar novamente para a esquerda.

SE (NÃO HOUVER CARROS SE APROXIMANDO E O SINAL ESTIVER ABERTO PARA PEDESTRES):

PASSO 5: Atravessar a rua em linha reta.

SENÃO:

PASSO 5: Esperar na calçada.

PASSO 6: Repetir do PASSO 2.

PASSO 7: Chegar ao outro lado da rua com segurança.

Check traffic



Traffic light

No – wait



Yes – cross



Is it safe to cross?

Look both ways

Watch turning cars

Keep eye contact



Por que o Pensamento Computacional Importa para Pesquisadores?



Além da Programação

O Pensamento Computacional, segundo Jeannette Wing, é uma habilidade analítica universal — não restrita a cientistas da computação.



Aplicado à Sua Pesquisa

Modelar problemas de biociências, negócios ou psicologia com lógica algorítmica torna a pesquisa mais rigorosa e reprodutível.



Colaboração e Confiança

Aprender em ambiente seguro e colaborativo derruba barreiras sociotécnicas e democratiza o acesso às ferramentas computacionais.

Média de Aluno: Calcule o Conceito!

Prepare-se para aplicar os pilares do Pensamento Computacional na prática! Seu desafio é criar um algoritmo que simule o cálculo e a atribuição de conceito para um aluno, com base em duas notas.

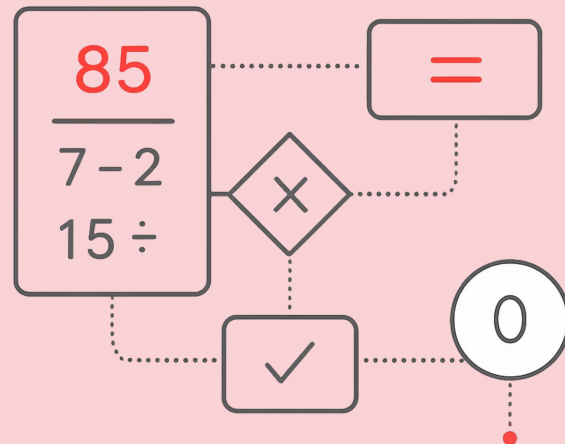
Seu algoritmo deve:

- Pedir ao usuário duas notas.
- Calcular a **média aritmética** dessas duas notas.
- Informar a média calculada.
- Atribuir um **conceito educacional** com base na média, seguindo estas regras:



- Média entre 0,0 e 5,0: RUIM
- Média maior que 5,0 e menor que 7,0: AINDA NÃO SUFICIENTE (ANS)
- Média igual ou maior que 7,0 e menor que 8,5: BOM
- Média igual ou maior que 8,5 e menor que 9,5: ÓTIMO
- Média igual ou maior que 9,5: EXCELENTE

Recomendação: Antes de qualquer linha de código ou ferramenta, descreva em linguagem natural (como português) a sequência lógica de passos para a solução. Mapeie o que **entra**, como será o **processo** e o que é a **saída**.



Média de Aluno: Calcule o Conceito!

Prepare-se para aplicar os pilares do Pensamento Computacional na prática! Seu desafio é criar um algoritmo que simule o cálculo e a atribuição de conceito para um aluno, com base em duas notas.

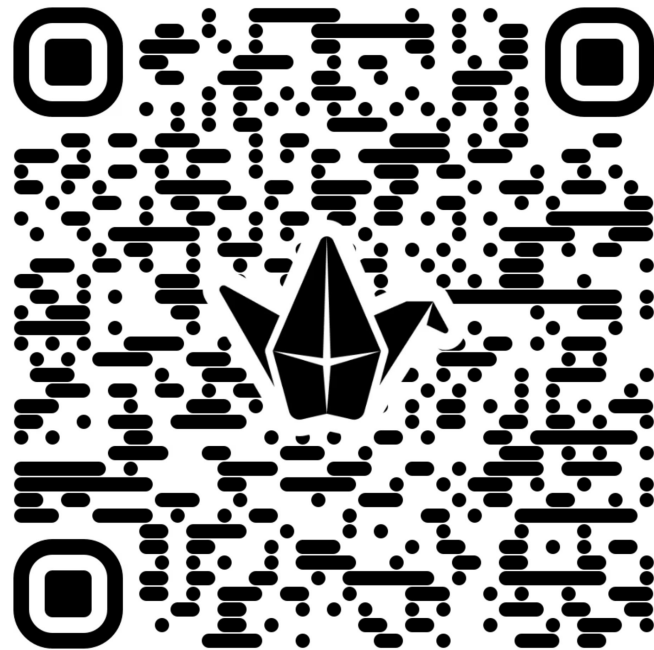
Seu algoritmo deve:

- Pedir ao usuário duas notas.
- Calcular a média aritmética dessas duas notas.
- Informar a média calculada.
- Atribuir um conceito educacional com base na média, seguindo estas regras:



- Média entre 0,0 e 5,0: RUIM
- Média maior que 5,0 e menor que 7,0: AINDA NÃO SUFICIENTE (ANS)
- Média igual ou maior que 7,0 e menor que 8,5: BOM
- Média igual ou maior que 8,5 e menor que 9,5: ÓTIMO
- Média igual ou maior que 9,5: EXCELENTE

Recomendação: Antes de qualquer linha de código ou ferramenta, descreva em linguagem natural (como português) a sequência lógica de passos para a solução. Mapeie o que **entra**, como será o **processo** e o que é a **saída**.



Vamos analisar o que vocês construíram

Após a tentativa de descrever o algoritmo de calcular a média e o conceito, vamos discutir a experiência em grupo:



Dificuldades

Quais foram os principais obstáculos encontrados ao tentar detalhar cada passo para o "computador"?



Pontos Ambíguos

Que partes da descrição ficaram abertas a diferentes interpretações ou careceram de clareza?



Acertos

Onde a lógica aplicada foi precisa e o "algoritmo", mesmo que inicial, se mostrou eficaz?

MÉTODO DE PARSONS

Método Parsons: Aprender sem Medo

O Método Parsons atua como um "andaime" (scaffold) cognitivo: o foco é puramente no sequenciamento lógico, eliminando o medo da sintaxe complexa e permitindo que o aluno raciocine sobre estrutura.

Monte algoritmos organizando blocos — sem digitar uma linha de código.



O que é?

Blocos de código embaralhados que você organiza na ordem certa.



Como funciona?

Arraste os blocos e monte a lógica. O foco é no raciocínio, não na sintaxe.



Por que funciona?

Menos estresse, mais aprendizado. Você pensa no problema — não no código.

Benefícios Cognitivos do Método Parsons

O Método Parsons oferece vantagens significativas para o processo de aprendizagem, focando em otimizar a cognição e reduzir barreiras.

Redução da Carga Cognitiva

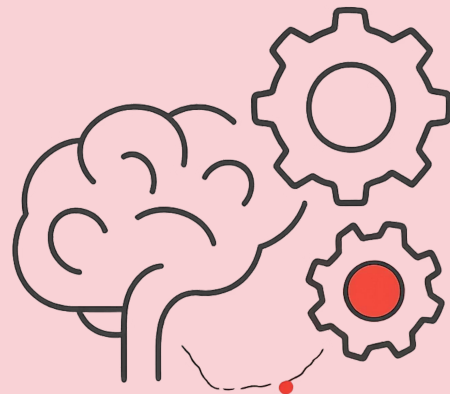
O cérebro concentra-se na lógica e resolução do problema, sem o estresse de sintaxe ou erros de digitação.

Foco no Pensamento Computacional

Ajuda a compreender como estruturas de controle (loops, condicionais) se encaixam e dependem umas das outras.

Feedback Imediato

Facilita a identificação de inconsistências, permitindo a auto-correção rápida e eficaz do raciocínio.



Aprender Junto Reduz a Barreira

A colaboração em duplas não é apenas uma estratégia pedagógica — é uma prática real da indústria de tecnologia. Ao trabalhar em par, os alunos ganham mais clareza, mais apoio e mais segurança para enfrentar o erro.



Driver & Navigator

Um escreve o código, o outro revisa em tempo real. Os papéis se alternam — e o aprendizado se multiplica.



Comunicação Técnica

Explicar o raciocínio em voz alta força a clareza do pensamento. Quem ensina, aprende duas vezes.



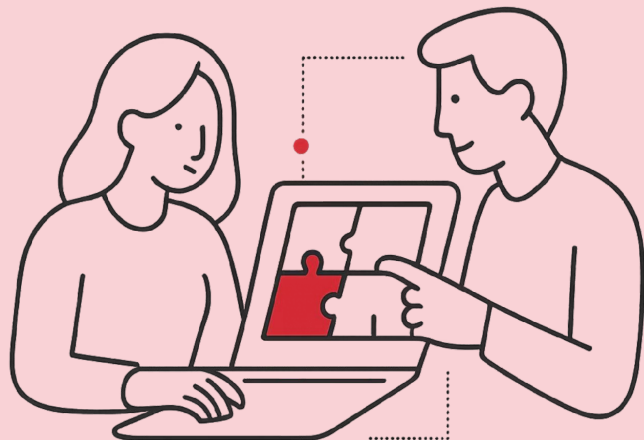
Confiança para Errar

Com um parceiro ao lado, o medo do erro diminui. O erro vira ponto de partida, não de chegada.



Mestre & Aprendiz

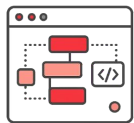
Ao relaizar a atividade em pares, um pode conduzir o processo enquanto o outro aprende.



Na indústria, ninguém programa sozinho. Na sala de aula, também não precisa.

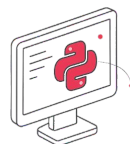
Implementação Visual com Google Blockly

O Google Blockly (<https://docs.blockly.com/>) é uma ferramenta gratuita e open-source do Google que permite programar arrastando e encaixando blocos coloridos — sem digitar uma linha de código. É a ponte perfeita entre o raciocínio lógico e a programação real.



Digitalizar a Lógica

Os alunos acessam o Google Blockly para implementar visualmente a lógica estruturada na Planilha Algorítmica. Cada bloco representa uma instrução: condicionais, loops, variáveis e operações matemáticas — tudo com feedback visual imediato. Não há erro de sintaxe possível: se o bloco encaixa, o código é válido.



Metacognição em Tempo Real

Ao alternar as abas da ferramenta, os alunos veem a tradução automática dos blocos visuais para Python, JavaScript ou outras linguagens. Esse momento é poderoso: o aluno percebe que já estava 'programando' sem saber. A abstração se torna concreta.



Blocos Encaixáveis

Cada bloco é uma instrução. Encaixou, funcionou. Sem erros de digitação, sem frustração.



Tradução Automática

O código gerado pode ser exportado em Python, JavaScript ou Dart — pronto para uso real.



100% Online e Gratuito

Sem instalação. Funciona direto no navegador, em qualquer dispositivo.



GOOGLE BLOCKLY

Mãos na Massa com Blockly: Seu Playground de Lógica!

Chegou a hora de transformar teoria em prática! Visite o ambiente de demonstração do Google Blockly. Aqui, você pode arrastar e conectar blocos coloridos para dar vida aos seus algoritmos, aplicando tudo o que aprendeu sobre decomposição, reconhecimento de padrões e abstração de forma divertida e sem complicação.

É um espaço seguro para experimentar, errar e aprender — sem se preocupar com a sintaxe. Clique e comece a construir sua primeira linha de lógica visual!

[Experimente o Blockly Demo](#)

 Ainda está com dúvidas? Lembre-se do Método Parsons: foque na lógica e na sequência dos blocos. A ferramenta vai te ajudar com o resto!

Desafio Blockly: Calculando os Degraus da Escada

Agora que você já explorou o Google Blockly, é hora de colocar o pensamento computacional em ação! Elabore um algoritmo para resolver o seguinte problema:

Crie um programa que receba a altura de um degrau de uma escada e a altura total que um usuário deseja alcançar.

Seu algoritmo deve calcular e exibir quantos degraus o usuário precisará subir para atingir seu objetivo. Não se preocupe com a altura do usuário neste cálculo.



Entradas

- Altura do degrau (ex: 15 cm)
- Altura desejada (ex: 300 cm)



Processamento

Calcule o número de degraus para alcançar a altura desejada.



Saída

Mostre o total de degraus necessários.

Lembre-se: pense na sequência lógica dos passos e utilize os blocos do Blockly para representar cada operação.

[Ir para o Blockly Demo](#)

Desafio Blockly: Média de Aluno – Calcule o Conceito!

Prepare-se para aplicar os pilares do Pensamento Computacional na prática! Seu desafio é criar um algoritmo que simule o cálculo e a atribuição de conceito para um aluno, com base em duas notas.

Seu algoritmo deve seguir estas etapas essenciais:



Coleta de Notas

Solicitar ao usuário as duas notas do aluno.



Cálculo da Média

Computar a média aritmética das notas fornecidas.



Exibição de Resultados

Apresentar a média calculada.



Atribuição de Conceito

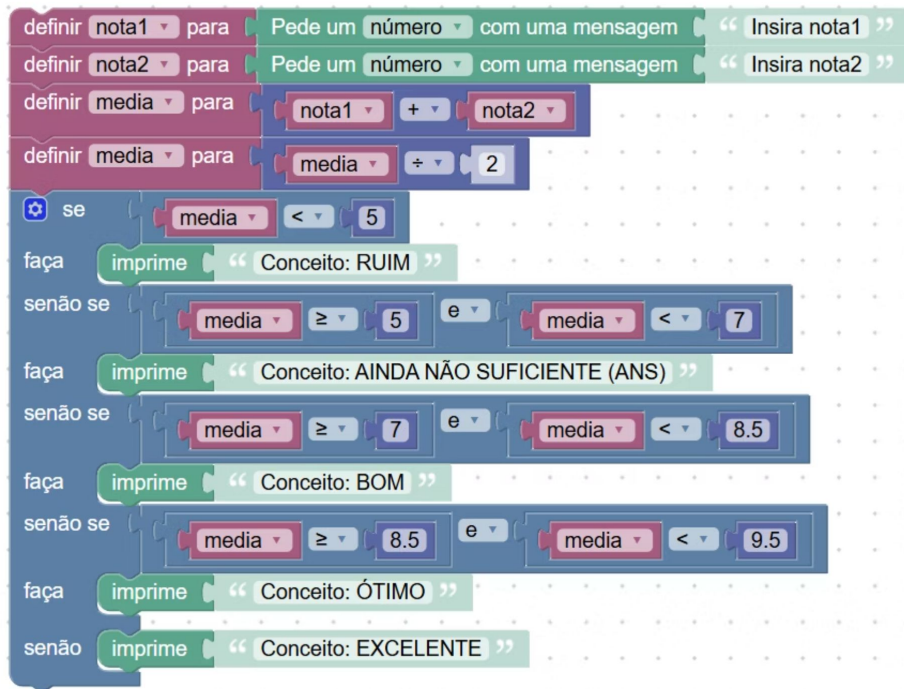
Definir o conceito educacional baseado na média.

Utilize a seguinte tabela para a atribuição de conceitos:

Média	Conceito
0,0 a 5,0	RUIM
> 5,0 e < 7,0	AINDA NÃO SUFICIENTE (ANS)
$\geq 7,0$ e < 8,5	BOM
$\geq 8,5$ e < 9,5	ÓTIMO
$\geq 9,5$	EXCELENTE

[Começar no Blockly Demo](#)

Desafio Blockly: Média de Aluno – Calcule o Conceito!



PRÁTICA

Desafio Blockly: Tabuada Interativa!

Agora iremos elaborar juntos um algoritmo que irá pedir ao usuário um número de 1 a 10, e o algoritmo irá calcular e exibir a tabuada desse número. Prepare-se para criar um algoritmo que vai transformar a matemática em um jogo!



Solicitar Entrada

Peça ao usuário que digite um número inteiro entre 1 e 10.



Calcular Tabuada

Multiplique o número inserido por cada inteiro de 1 a 10.



Exibir Resultados

Mostre cada linha da tabuada de forma organizada.

Pense nos blocos de "repetição" e "exibir" para construir a sequência lógica que resolve o desafio.

[Começar no Blockly Demo](#)

Desafio Blockly: Tabuada Interativa!

The image shows the Blockly IDE interface with a JavaScript script for an interactive multiplication table. The script is as follows:

```
var numero, i, texto;

numero = Number(window.prompt('Digite o número:'));
for (i = 1; i <= 10; i++) {
  texto = [texto, i, ' x ', numero, ' = ', i * numero].join('');
}
window.alert(texto);
```

The Blockly workspace on the left shows the visual representation of this script. It includes a 'set' block for 'numero' to a 'prompt for number' block with the message 'Digite o número:'. This is followed by a 'count with' block from 1 to 10 by 1. A 'do' loop contains a 'set' block for 'texto' to a 'create text with' block. The 'create text with' block is composed of several blocks: 'texto', 'i', 'x', 'numero', '=', and 'i x numero'. The 'do' loop is followed by a 'print' block for 'texto'.



PRÁTICA

Avaliação Blockly



É hora de avaliar nossa prática!

Acesse o formulário pelo QR Code e responda às questões sobre o que você praticou no Blockly.

Introdução a uma Linguagem de Programação



Agora que já entendemos o Pensamento Computacional e experimentamos soluções no Google Blockly, chegou a hora de ver essas ideias em código escrito. Nesta etapa, vamos comparar os blocos visuais com a lógica de uma linguagem real, como o Python.

Python é uma ótima primeira linguagem porque tem sintaxe simples e fácil de ler. Além disso, oferece muitos recursos para aprender e aplicar em áreas como automação, ciência de dados e inteligência artificial.

Sintaxe clara

Facilita a leitura e ajuda a focar na lógica do algoritmo.

Ampla comunidade

Oferece suporte, exemplos e materiais de estudo para iniciantes.

Muitas aplicações

É usado em ciência de dados, pesquisa, automação e desenvolvimento.

Variáveis: Onde Guardamos Nossas Informações

No mundo da programação, as **variáveis** são como caixas rotuladas onde podemos armazenar diferentes tipos de informações. Elas permitem que você nomeie e manipule dados, tornando seu código dinâmico e fácil de entender.

Em Python, criar uma variável é muito simples. Você apenas dá um nome a ela e atribui um valor:

```
# Variável para armazenar um nome
nome_aluno = "Maria"

# Variável para armazenar uma idade
idade = 30

# Variável para armazenar um valor booleano
esta_aprovado = True
```

Aqui, `nome_aluno`, `idade` e `esta_aprovado` são as variáveis. O sinal de igual (=) é o operador de atribuição, que coloca o valor à direita dentro da "caixa" nomeada à esquerda. Cada variável pode guardar um tipo diferente de informação, como texto (string), números inteiros (integer) ou valores verdadeiros/falsos (booleanos).



Dica de Nomenclatura: Em Python, é uma boa prática usar nomes descritivos para suas variáveis, separados por sublinhados (snake_case), como `quantidade_total` ou `preco_unitario`.

Operadores Matemáticos: Aritmética no Código

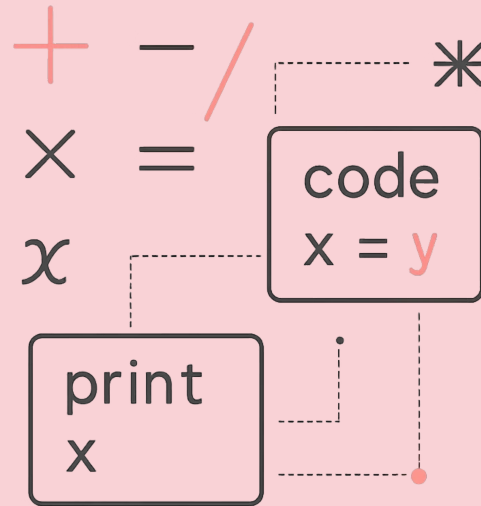
No mundo da programação Python, os operadores matemáticos são suas ferramentas para realizar cálculos. Eles permitem que seu programa execute operações aritméticas como adição, subtração, multiplicação e divisão, transformando dados numéricos em novas informações.

Compreender esses operadores é fundamental para construir qualquer lógica que envolva processamento de números.

Exemplos de Operadores Aritméticos

```
soma = 10 + 5          # Adição
subtracao = 20 - 7     # Subtração
multiplicacao = 4 * 6  # Multiplicação
divisao = 15 / 3       # Divisão (retorna número decimal)
divisao_inteira = 15 // 4 # Divisão inteira (descarta decimais)
resto = 17 % 5         # Módulo (resto da divisão)
potencia = 2 ** 3      # Potenciação (2 elevado a 3)
```

```
# print(soma) # Saída: 15
# print(divisao) # Saída: 5.0
# print(resto) # Saída: 2
```



Estruturas Condicionais: Tomando Decisões no Código

No coração de todo programa inteligente está a capacidade de tomar decisões. As estruturas condicionais em Python permitem que seu código execute diferentes blocos de instruções com base em se uma condição é verdadeira ou falsa.

Com elas, seu programa pode reagir a diferentes entradas, estados ou valores, tornando-o dinâmico e flexível. Veja alguns exemplos:

```
# Exemplo 1: Verificando a idade
idade = 20

if idade >= 18:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")

# Exemplo 2: Classificando notas
nota_final = 8.2

if nota_final >= 9.0:
    print("Conceito: Excelente!")
elif nota_final >= 7.0:
    print("Conceito: Bom!")
elif nota_final >= 5.0:
    print("Conceito: Regular.")
else:
    print("Conceito: Insuficiente.")
```

Em Python, as palavras-chave `if`, `elif` (abreviação de "else if") e `else` são usadas para definir essas condições. Cada condição é avaliada em sequência, de cima para baixo. Se uma condição for verdadeira, o bloco de código indentado abaixo dela é executado, e o restante da estrutura condicional é ignorado.



Importante: A **indentação** (espaços ou tabs no início da linha) é fundamental em Python! Ela não é apenas estética; define os blocos de código pertencentes a cada parte da estrutura condicional.

Estruturas de Repetição: Ações Que se Repetem

Em programação, muitas vezes precisamos executar a mesma ação múltiplas vezes. É aí que entram as estruturas de repetição, também conhecidas como loops. Elas permitem que seu código execute um bloco de instruções repetidamente, economizando tempo e tornando o código mais eficiente e conciso.

Python oferece principalmente dois tipos de loops: o `for` e o `while`.



Exemplo 1: Loop 'for' (para um número conhecido de repetições ou sequências)

```
print("Contagem regressiva com 'for':")
for i in range(5, 0, -1): # i vai de 5 até 1 (não inclui o 0), decrementando 1
    print(i)
print("Decolar!")
```

Exemplo 2: Loop 'while' (enquanto uma condição for verdadeira)

```
contador = 0
print("\nContagem progressiva com 'while':")
while contador < 3: # Repete enquanto contador for menor que 3
    print(f"O contador é {contador}")
    contador = contador + 1 # Incrementa o contador para evitar loop infinito
print("Fim da contagem!")
```

O loop `for` é ideal quando você sabe quantas vezes quer repetir algo ou quando precisa iterar sobre os itens de uma sequência (como uma lista ou um intervalo de números). Já o loop `while` é mais adequado quando você quer que o código se repita enquanto uma determinada condição for verdadeira, sem saber de antemão o número exato de repetições.



Cuidado com Loops Infinitos! Ao usar `while`, certifique-se de que a condição se tornará falsa em algum momento, ou seu programa pode entrar em um "loop infinito", consumindo recursos e travando.

Arremate: Praticando Python Básico

Agora é hora de colocar em prática o que você aprendeu sobre variáveis, operadores, condicionais e loops. Complete o exercício abaixo e teste diferentes valores para ver como o programa se comporta.

```
nome = input("Digite o nome do aluno: ")
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))
nota3 = float(input("Digite a terceira nota: "))

media = (nota1 + nota2 + nota3) / 3

print("Aluno:", nome)
print("Média:", media)

# complete aqui: crie a estrutura if/elif/else para mostrar o conceito
# A: média maior ou igual a 9
# B: média maior ou igual a 7
# C: média maior ou igual a 5
# D: média menor que 5

if media >= 9:
    print("Conceito A")
# complete aqui
# complete aqui
# complete aqui
```

Variáveis e tipos de dados

Use `input()` para receber texto e `float()` para transformar notas em números decimais.

Estrutura condicional

Organize as condições com `if`, `elif` e `else` para escolher o conceito correto.

Teste com valores diferentes

Experimente notas altas, médias e baixas para verificar se o resultado muda como esperado.

ARREIMATE

O que Construimos Hoje

Avançamos do pensamento computacional à programação visual e textual, conectando lógica, prática e confiança para transformar ideias em soluções.

Pensamento Computacional

Exploramos os 4 pilares — decomposição, reconhecimento de padrões, abstração e algoritmos — como base para resolver problemas.

Prática com Blockly

Usamos algoritmos visuais para organizar etapas, testar soluções e entender a lógica antes do código.

Método Parsons

Trabalhamos a lógica sem medo, reorganizando blocos e fortalecendo a aprendizagem com apoio gradual.

Python Básico

Introduzimos variáveis, operadores matemáticos, condicionais e estruturas de repetição para dar os primeiros passos na programação.

Próximos Passos: Continue Aprendendo

Continue praticando fora da sala de aula: cada pequena atividade ajuda a fortalecer sua lógica, sua autonomia e sua confiança para aprender programação.

CS Unplugged

Atividades desplugadas para reforçar os 4 pilares do Pensamento Computacional de forma lúdica e acessível. csunplugged.org

Google Blockly Games

Jogos interativos para praticar lógica de programação visual, ideal para consolidar o que foi visto em aula. blockly.games

Python.org — Tutorial Oficial

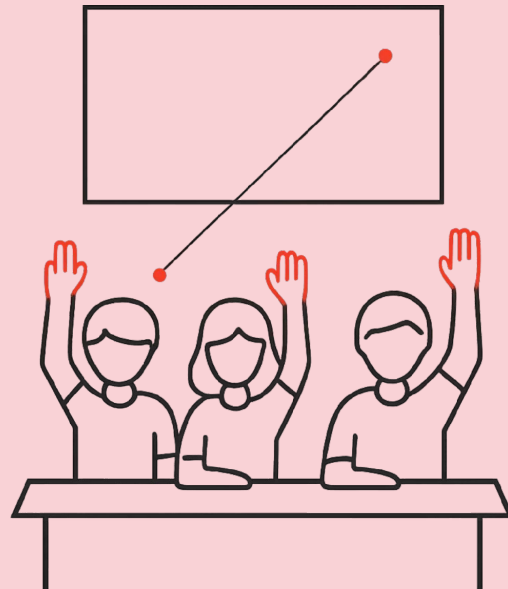
Documentação oficial em português com exemplos práticos de variáveis, condicionais e loops. docs.python.org/pt-br/3/tutorial

Exercism — Python Track

Plataforma gratuita com exercícios progressivos de Python com feedback de mentores. exercism.org/tracks/python

Codecademy — Learn Python 3

Curso interativo e gratuito para praticar Python no navegador, do básico ao intermediário. codecademy.com



Introdução ao Pensamento Computacional

UFPE - Centro de Informática - IEC - 2026.1

DURAÇÃO: 4 HORAS

GOOGLE BLOCKLY

